AI-Library — Architecture Guide for Backend Developers

Audience: Backend developers learning Microsoft.Extensions.AI
Stack: .NET 10 Web API · Angular 19 · Ollama · MediatR
Learn path: https://learn.microsoft.com/en-gb/dotnet/ai/quickstarts/build-chat-app?pivots=openai
Date: August 2026

> **Purpose of this PDF**
> Explain every major part of the backend and Angular frontend, how they connect, and how each piece maps to the Microsoft Learn "build a chat app" quickstart — adapted into an *AI-powered library* product instead of a console hiking bot.

1. Big picture

```
Browser (Angular :4200)
      |  JSON POST /api/chat
      |  or SSE  POST /api/chat/stream
      v
AiLibrary.Api  (:5243)
      |  MediatR ChatCommand / StreamChatCommand
      v
AiLibrary.Application  (use cases + ports)
      |
      +-- IChatHistoryStore  -->  in-memory sessions + system prompt
      +-- IChatService       -->  IChatClient (Ollama via MEAI)
      v
Ollama (:11434)  model e.g. gemma3:4b
```

The Microsoft sample is a **console loop**. This project keeps the same AI concepts but exposes them as an **HTTP API** and a **web UI** so you can learn production-shaped structure.

2. Backend parts (what each project is for)

Project	Role	Key types
`AiLibrary.Api`	Composition root + HTTP edge. No business logic beyond mapping HTTP ↔ commands.	`Program.cs`, `ChatController`, CORS, OpenAPI, `Http/*.http`
`AiLibrary.Application`	Use cases (CQRS-style via MediatR). Defines ports (interfaces) the infrastructure implements.	`ChatCommand`, `StreamChatCommand`, `ChatRequest/Response`, `IChat*`
`AiLibrary.Infrastructure`	Real implementations: Ollama client, history store, librarian system prompt.	`ChatService`, `ChatHistoryStore`, `PromptBuilder`, `AddAiServices`
`AiLibrary.Domain`	Reserved for library domain (Book, Author…) later. Empty on purpose until catalog/RAG.	—

2.1 Program.cs — host

- Registers AI services (`AddAiServices`), MediatR, controllers, CORS for Angular `localhost:4200`.
- Maps controllers; enables OpenAPI in Development.
- **Why:** Same job as the top of the Learn sample's `Program.cs`, but for ASP.NET Core instead of a console.

2.2 ChatController — HTTP surface

Endpoint	Behavior	Angular usage
`POST /api/chat`	Full reply as JSON `{ sessionId, message }`	Default / "Stream replies" off — `HttpClient.post`
`POST /api/chat/stream`	SSE: `session` → many `token` → `done`	"Stream replies" on — `fetch` + ReadableStream

Both paths share the same history store. Streaming is the Learn sample's `GetStreamingResponseAsync` loop, moved behind HTTP.

2.3 Application commands

- `ChatCommand` → non-streaming: add user msg → `CompleteAsync` → save assistant msg → return DTO.
- `StreamChatCommand` → add user msg → yield tokens → save full assistant text when stream ends.

Why MediatR: keeps the controller thin and makes it easy to add logging/validation behaviors later without cluttering HTTP code.

2.4 IChatClient + ChatService

Microsoft.Extensions.AI gives you `IChatClient` so application code does not depend on Ollama or OpenAI types.

```
// Infrastructure registration (conceptual)
services.AddSingleton<IChatClient>(_ =>
    new OllamaChatClient(new Uri(endpoint), model));

// Learn sample (OpenAI pivot) would instead do:
// new OpenAIClient(key).GetChatClient(model).AsIChatClient();
```

`ChatService` wraps:

- `GetResponseAsync` → complete message (JSON endpoint)
- `GetStreamingResponseAsync` → token stream (SSE endpoint)

2.5 ChatHistoryStore — multi-turn memory

Learn sample: one `List<ChatMessage>` for the whole console process.

This app: `ConcurrentDictionary<sessionId, messages>` with per-session locks.

- New session → insert librarian **system** message once.
- Each turn → append user, call model with full history, append assistant.
- Trim old turns if the list grows past a cap (keeps system message).

Important for backend devs: history dies when the API process restarts (in-memory). That is intentional for the learning stage before DB/RAG.

2.6 PromptBuilder — librarian persona

Maps 1:1 to the hiking system prompt in the Learn article. Here the persona is **Ava, librarian**: ask genre/level/mood, recommend up to three titles, spoiler-safe summaries, remember prior turns.

2.7 Configuration

```
appsettings.json → Ollama:Endpoint, Ollama:Model
UserSecretsId on Api project → ready if you switch to OpenAI keys later
```

3. Frontend parts (Angular) — what a backend dev should know

Path	Role
`AI-Library app/src/environments/`	`apiBaseUrl` pointing at the API (default `http://localhost:5243`)
`core/models/chat.models.ts`	Mirrors backend DTOs (`sessionId`, `message`)
`core/services/chat.service.ts`	HTTP client for JSON + fetch-based SSE parser
`features/chat/chat.component.*`	UI: messages, session id, stream toggle, new chat
`app.config.ts`	`provideHttpClient()` + router

3.1 How the UI keeps multi-turn state

1. First send: no `sessionId` → API creates one → UI stores it.
2. Later sends: UI posts the same `sessionId`.
3. **New chat** clears local `sessionId` only; server still has old sessions in memory until restart (harmless).

The **source of truth for history is the backend store**, not Angular. The UI only displays bubbles and holds the id.

3.2 Why fetch for streaming?

Browser `EventSource` is GET-only. Our stream endpoint is POST (body carries the message), so Angular uses `fetch` + reads SSE lines (`event:` / `data:`).

3.3 CORS

Angular origin `http://localhost:4200` ≠ API origin `http://localhost:5243`. `Program.cs` allows that origin so browsers accept API responses. Backend tools (.http, curl) ignore CORS.

4. Request/response contracts

```
// Request
{ "sessionId": "optional-string", "message": "required string" }

// JSON response
{ "sessionId": "…", "message": "assistant full text" }
```

```
// SSE events
event: session
data: {"sessionId":"…"}

event: token
data: {"text":"partial"}

event: done
data: {"sessionId":"…"}
```

5. Microsoft Learn quickstart — side-by-side match

Learn quickstart	AI-Library equivalent	Notes
Console project	Web API + Angular	Same AI ideas; better product shape
Packages: OpenAI + MEAI.OpenAI	MEAI + MEAI.Ollama	Abstraction is the lesson; provider is swappable
User secrets OpenAIKey / ModelName	appsettings Ollama endpoint/model	UserSecretsId already on Api for later OpenAI
`IChatClient` from OpenAIClient	`OllamaChatClient` as `IChatClient`	Handler/service code stays provider-agnostic
Hiking system prompt	Librarian system prompt (`PromptBuilder`)	Same technique, different domain
`List<ChatMessage> chatHistory`	`ChatHistoryStore` by sessionId	Multi-user / multi-tab capable (in-memory)
`while(true) ReadLine`	Angular send loop + `.http` multi-turn scenarios	Interactive + automated-friendly
`GetStreamingResponseAsync` to console	SSE tokens to browser	Same stream API, different transport

> **Learning takeaway:** If you can point to system prompt, history list, IChatClient, and streaming in both the Learn article and this repo, you have completed the quickstart's core skills — and already layered them for a real library assistant.

6. End-to-end flows

6.1 JSON multi-turn (recommended first)

1. UI/http → `POST /api/chat` with message only.
2. Handler creates sessionId, history gets system+user.
3. Ollama completes; assistant saved; JSON returned.
4. Client resends sessionId with "similar books" / "summary".
5. Model sees full prior transcript → coherent multi-turn.

6.2 Streaming

1. Same history append for user message.

2. Tokens flushed as SSE `token` events.
3. On completion, full text stored as assistant message (so the next turn still works).

7. How to test (backend-first)

- File: `src/AiLibrary.Api/Http/chat.http` — scenarios A–I with comments.
- Manual multi-turn script: Hobbit → similar books → summary of first recommendation.
- Angular: run both processes; watch session id under the header; toggle stream.

8. What we deliberately postponed

- SQL/catalog database for books
- RAG / embeddings
- Auth, rate limits, durable session store

Order recommended by the product path: solid chat + history → catalog/tools → RAG.

9. File map (quick navigation)

```
Backend
  Program.cs                          host, CORS, DI
  Controllers/ChatController.cs       HTTP endpoints
  Commands/ChatCommandHandler.cs      non-stream use case
  Commands/StreamChatCommandHandler   stream use case
  Services/ChatService.cs             IChatClient wrapper
  Services/ChatHistoryStore.cs        session memory
  Services/PromptBuilder.cs           librarian system prompt
  Http/chat.http                      executable API tests

Frontend
  core/services/chat.service.ts       API client
  features/chat/chat.component.*      UI
  environments/environment*.ts        apiBaseUrl
```

10. Setup cheat sheet

```
ollama pull gemma3:4b
dotnet run --project src/AiLibrary.Api --launch-profile http
cd "AI-Library app" && npm start
# API http://localhost:5243   UI http://localhost:4200
```

AI-Library learning record · Compare with Microsoft Learn quickstart “Build an AI chat app with .NET” · Provider today: Ollama · Abstraction: Microsoft.Extensions.AI